

COMPUTER PROGRAMMING MID-1 QUESTION BANK

PART-A (SHORT ANSWER TYPE QUESTIONS):

1. Define software and programming.

Ans:

SOFTWARE:

It is basically a set of instructions or commands that tell a computer to do a specific task that serves its users.

Examples are MS-Word, MS-Excel, PowerPoint and Web Browser.

PROGRAMMING:

Programming is the process of designing, writing, testing, and maintaining a set of instructions (called code) that a computer follows to perform a specific task.

These instructions are written using programming languages like C, Python, Java, etc.

2. What is type conversion? Give one example.

Ans:

Type Conversion in Python refers to the process of converting one data type into another. This is often necessary when performing operations that require data to be in a specific format.

- Convert the data type of an object to another.

Syntax: **datatype(object)**

- Changes can be stored in same variable or in different variable.

Example:

```
a=23.65
b=int(a)
print(type(b))
```

```
<class 'int'>
```

3. Define syntax in python.

Ans: The set of guidelines that specify the organization and sequencing of statements in a programming language are referred to as “syntax”. The python syntax is a collection of guidelines that define how the language can be expressed and used. When writing code, Python syntax comprises keywords, operators, expressions, statements.

4. Define variable with an example in Python.

Ans: A variable in Python is a symbolic name that acts as a reference or container for storing data values. Values assigned to variables using an assignment operator '='. Variables can be named alphanumerically. However, the first letter must start with an alphabet (lowercase, uppercase).

Example: abc=12, a_bc=13, ab1=23.....

5. List any four arithmetic operators in Python.

Ans: Arithmetic operators are used to perform mathematical operations between two operands.

SYMBOL	OPERATION
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponent
//	Double division

6. What is the difference between = and == operators?

Ans: In Python, = and == are fundamentally different operators with distinct purposes:

1. = (Assignment Operator):

- Used to assign a value to a variable.
- It does not compare values; it simply stores the value on the right-hand side into the variable on the left-hand side.

Example:

a = 5 # Assigns the value 5 to the variable a

b = "Hello world" # Assigns the string "Hello" to the variable b

2. == (Equality Operator):

- Used to compare two values for equality.
- Returns True if the values on both sides are equal, otherwise returns False.

Example:

a = 10

```
print(a == 10) # True, because x is equal to 10
print(a == 5)  # False, because x is not equal to 5
```

7. What is a data type in programming.

Ans: The data types are used to define the type of a variable. It represents the type of data we are going to store in a variable and determines what operations can be done on it.

Syntax: type(variable name)

8. Write short notes on python input-output functions.

Ans: With the print() function, we can display output in various formats, while the input() function enables interaction with users by gathering input during program execution.

Taking input in Python:

Python's input() function is used to take user input. By default, it returns the user input in form of a string.

Example:

```
name = input("Enter your name: ")
print(name)
```

```
Enter your name: Hello world
Hello world
```

Output Function:

We use the print() function or print keyword to output data to the standard output device (screen). This function prints the object/string written in function.

```
print("Hello, World!")
```

```
Hello, World!
```

9. Write the syntax of an if statement in Python.

Ans: An if statement is used as a form of conditional programming; something you probably do every day in the real world.

In its most basic form, the if statement is

```
if <condition-evaluating-to-boolean>:  
    statement
```

It is important that the condition must evaluate to True or False. If the condition is True, then it will execute the indented statement.

10. What is the difference between local and global variables?

Ans:

Comparison	Local Variable	Global Variable
Definition	Local variables are declared within the functions.	Global variables are declared outside the functions.
Scope	Accessible inside the function.	Accessible throughout the code.
Parameter Passing	For local variables, parameter passing is necessary.	For global variables, parameter passing is not necessary.
Naming Restrictions	Different variables with the same name can be declared in different functions.	Global variables should be uniquely named i.e. many variables with the same name cannot exist.

11. Define return function with an example.

Ans: In Python, whenever a return statement is encountered within a function then that function will terminate and return any values following the return keyword.

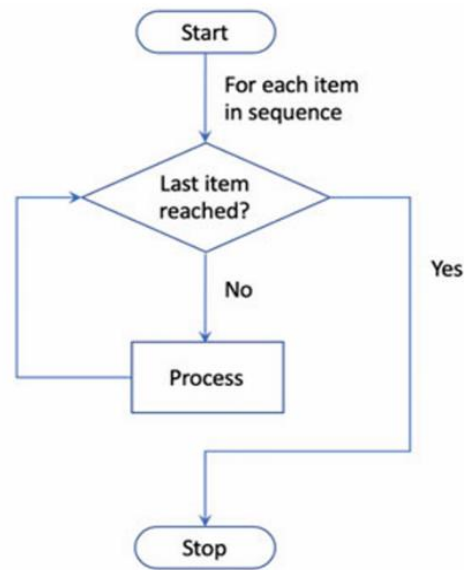
For example, the following defines a simple function that squares whatever value has been passed to it: When we call this function, it will multiple whatever it is given by itself and then return that value.

```
def square(n):  
    return n * n  
square(45)
```

2025

12. What is a for loop in Python?

Ans: The **for** loop is used to step a **variable** through a series of values until a given test is met.



13. Define the terms Parameters and Arguments.

Ans:

Parameter: A parameter is a variable defined as part of the function header and is used to make data available within the function itself.

Argument: An argument is the actual value or data passed into the function when it is called. The data will be held within the parameters.

14. What is the use of the elif keyword?

Ans: In some cases, there may be several conditions you want to test, with each condition being tested if the previous one failed. This else-if scenario is supported in Python by the elif element of an if statement. The elif element of an if statement follows the if part and comes before any (optional) else part.

15. Define a function in Python.

Ans: In Python, Functions are groups of related statements that can be called together, that typically perform a specific task, and which may or may not take a set of parameters or return a value. Functions can be defined in one place and called or invoked in another. This helps to make code more easier to understand.

16. What is isdigit() string method?

Ans: The **isdigit()** method in Python is a built-in string method that checks whether all the characters in a string are digits. It returns **True** if every character in the string is a **digit** (0-9). However, it also returns **True** for certain **Unicode characters**, such as

subscripts and superscripts (like ² and ₁), that are defined as digits. It returns **False** otherwise (if the string is empty or contains non-digit characters).

17. Define list in Python with example.

Ans: A List is an **ordered set of values**, where each value is **identified by an index**. The Values that make up a list are called its **elements or items**. Lists are **Mutable** and so their elements can be changed

Creating a list:

Syntax: Lists are created using square brackets [] to enclose elements, separated by commas.

For example:

```
list1 = ['John', 'Paul', 'George', 'Ringo']  
print(list1)
```

Output:

```
['John', 'Paul', 'George', 'Ringo']
```

18. Give an example for negative indexing in lists.

Ans: Negative Indexing: Access items from the end of the list using negative indices. -1 refers to the last item.

Example:

```
list1 = [23,'xyz',0.895, "ABC"]  
print('list1[-1]:', list1[-1])
```

Output: list1[-1]: ABC

19. What is extended slicing in Python?

Ans: Extended slicing in Python is a feature that allows sequences (like lists, tuples, and strings) to be accessed and manipulated using an additional parameter **the step value** beyond the traditional start and stop indices.

Syntax for extended slicing: sequence [start:stop:step]

20. Define tuple in python.

Ans: Tuples are an immutable ordered collection of objects; that is each element in a tuple has a specific position (its index) and that position does not change over time. Indeed, it is not possible to add or remove elements from the tuple once it has been created.

Usage:

Tuples are commonly used when:

- You want to store a fixed set of values
- You want to ensure data cannot be changed (immutability)
- You need to return multiple values from a function

PART-B (LONG ANSWER TYPE QUESTIONS):**1. Discuss why Python is widely used compared to other programming languages.**

Ans: The primary factors cited by Python users are:

- 1. Software quality:** Python code is designed to be **readable**, and hence, **reusable** and **maintainable** much more than traditional scripting languages. In addition, Python has deep support for more advanced software reuse mechanisms, such as object-oriented programming (OOP).
- 2. Developer productivity:** Python boosts developer productivity many times beyond compiled or statically typed languages such as C, C++, and Java. Python code is typically one-third to one-fifth the size of equivalent C++ or Java code.
- 3. Program portability:** Most Python programs run unchanged on all major computer platforms. Porting Python code between Linux and Windows, for example, is usually just a matter of copying a script's code between machines.
- 4. Support libraries:** Python comes with a large collection of prebuilt and portable functionality, known as the **standard library**. Python can be extended with both homegrown libraries and a vast collection of third-party application support software.
- 5. Component integration:** Python scripts can easily communicate with other parts of an application, using a variety of integration mechanisms. Such integrations allow Python to be used as a product customization and extension tool.
- 6. Enjoyment:** Because of Python's ease of use and built-in toolset, it can make the act of programming more pleasure than chore.

2. Define operator and classify its types. Write a short note on logical operators in Python with examples.

Ans: Operators are special symbols that help in carrying out an assignment operation or arithmetic or logical computation. Value that the operator operates on is called operand

Example: $a+b$ # here a and b are operands whereas '+' indicates operator.

Types of Operators:

1. Arithmetic Operator
2. Logical Operators
3. Assignment Operator
4. Comparison operator
5. Bitwise operator

❖ LOGICAL OPERATORS:

Logical operators are used when operands are conditional statements and returns Boolean value. In python, logical operators are designed to work with Boolean values. These operators include **and**, **or**, and **not**, each serving a specific purpose in logical operations.

1. **and**: Returns **True** if both conditions are **True**. If either condition is **False**, the result is **False**.

Example:

```
x=4
y=10
print((x<5) and (y>7))
```

True

```
x=4
y=10
print((x<5) and (y<7))
```

False

2. **or**: Returns **True** if at least one condition is **True**. If both conditions are **False**, the result is **False**.

Example:

```
x=4
y=10
print((x<5) or (y<7))
```

True

```
x=4
y=10
print((x>5) or (y<7))
```

False

3. **not**: Reverses the Boolean value of the condition. If the condition is **True**, it returns **False**, and vice versa.

Example:

```
x=False  
print(not x)
```

True

```
x=True  
print(not x)
```

False

3. Explain briefly about data types in python.

Ans: A data type defines the kind of value an object can hold and the operations that can be performed on it. Everything in Python is an object, and every object has a data type.

Python's data types are broadly categorized by

Object Type	Description	Example
Fundamental or Build-in Data types or Data Structures		
1. int	We can use to represent the whole/integral Numbers	A=26,b=10, C=-12, d=-26
2. float	We can use to represent the decimal/floating point numbers	A=26.6, b=-26.2
3. complex	We can use to represent the complex numbers	C=26+26j
4. bool	We can use to represent the logical values(Only allowed values are True and False)	X=True, y= False
5. str	To represent sequence of Characters	A="MRV", v="hello"
Composite Data Types or Data Structures		
6. range	To represent a range of values	r=range(26) r1=range(1,26) r2=range(1,2,3)
7. list	To represent an ordered collection of objects	L1=[1,2,3,4,5]
8. tuple	To represent an ordered collections of objects	t=(1,2,3,4,5)
9. set	To represent an unordered collection of unique Objects	S={1,2,3,4,5}
10. dict	To represent a group of key value pairs	d={1:'abc',2:'def',3:'xyz'}
11. None	None means Nothing or No value associated.	a=None

4. Explain briefly about Bitwise operators in python with examples.

Ans: Bitwise operators in Python are used to perform operations on binary representations of integers. These operators work at the bit level, manipulating individual bits of the operands.

BITWISE AND (&):

- **Bitwise And(&)** compares each bit of two numbers and returns 1 if both bits are 1, otherwise 0.

Example:

```
a=9
b=7
print(a & b)
```

1

BITWISE OR (|)

- **Bitwise Or(|)** compares each bit of two numbers and returns 1 if at least one of the bits is 1.

Example:

```
a=9
b=7
print(a | b)
```

15

BITWISE XOR (^)

- **Bitwise XOR** Compares each bit of two numbers and returns 1 if the bits are different, otherwise 0.

Example:

```
a=9
b=7
print(a ^ b)
```

14

BITWISE NOT (~)

- **Bitwise NOT** inverts all the bits of the number (1 becomes 0, and 0 becomes 1). It also changes the sign of the number due to two's complement representation.

Example:

```
a=9  
print(~a)
```

-10

BITWISE LEFT SHIFT (<<):

- **Bitwise Left Shift** shifts the bits of the number to the left by the specified number of positions, filling with 0s on the right.

Example:

```
a=9  
print(a<<2)
```

36

BITWISE RIGHT SHIFT (>>):

- **Bitwise Right Shift** shifts the bits of the number to the right by the specified number of positions, discarding bits shifted out.

Example:

```
a=9  
print(a>>1)
```

4

5. Explain variable declaration rules in Python with suitable examples.

Ans:

- A variable in Python is a symbolic name that acts as a reference or container for storing data values.
- Values assigned to variables using an assignment operator '='.

Example: a=12

- Variable name should be short and descriptive.
- Avoid using variable names that clash with inbuilt functions.

Example:

```
if=23  
print(if)
```

SyntaxError: invalid syntax

- Variables can be named alphanumerically.

Example: abc=12, a_bc=13, ab1=23.....

- However, the first letter must start with an alphabet (lowercase, uppercase).

Example: Abc=10, a=10

- Underscore (_) is the only special character used in variable declaration.

Example: even_odd=45

- Use of any other special character will throw an error.

Example:

```
a&c=15  
print(a&c)
```

Output:

SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of '='?

- Variable names should not begin or end with underscore even though both are allowed

Example: _age=55

ge_=55

- Assigning multiple values to multiple variables:

```
abc,cde,efg=10,20,30
print("abc=",abc)
print("cde=",cde)
print("efg=",efg)
```

Output:

```
abc= 10
cde= 20
efg= 30
```

6. Explain type conversions in detail.

Ans: Type Conversion in Python refers to the process of converting one data type into another. This is often necessary when performing operations that require data to be in a specific format.

Coercing object to new data type:

- Convert the data type of an object to another

Syntax: datatype(object)

- Changes can be stored in same variable or in different variable

```
Employee_name="Ram"
Age=55
Height=150.6
```

Coercing the data type of an object

```
height=150.6
ht= int(height)
print(type(ht))
```

<class 'int'>

```
height=int(height)
print(type(ht))
```

<class 'int'>

- Only few coercions are accepted
- Consider the variable 'Salary_tier' which is of string data type
- 'Salary_tier' contains an integer enclosed between single quotes.

```
Salary_tier='1'
```

Coercing the data type of an object

```
Salary_tier='1'
print(type(Salary_tier))
<class 'str'>

Salary_tier=int(Salary_tier)
print(type(Salary_tier))
<class 'int'>
```

- However, if the value enclosed within the quotes is a string then conversions will not be possible.

Example:

```
Employee_name='Ram'
a=float(Employee_name)
```

Output:

```
1 Employee_name='Ram'
----> 2 a=float(Employee_name)

ValueError: could not convert string to float: 'Ram'
```

7. a) Explain Operator Precedence in python

b) Simplify Expression $A = 7 - 2 * (27 / 3 ** 2) + 4$ using Operator Precedence.

Ans:

a) Operator Precedence in Python:

Decreasing order of precedence	Operation
Parentheses	()
Exponent	**
Division	/
Multiplication	*
Addition and subtraction	+, -
Bitwise AND	&
Bitwise OR	

Relational/ comparison operators	==, !=, >, >=, <=
Logical NOT	not
Logical AND	and
Logical OR	or

b) **Simplify Expression $A = 7 - 2 * (27 / 3 ** 2) + 4$ using Operator Precedence.**

Expression:

$$A = 7 - 2 \times \left(\frac{27}{3^2} \right) + 4$$

Step 1: Evaluate exponentiation 3^2

$$3^2 = 9$$

Expression becomes:

$$7 - 2 \times \left(\frac{27}{9} \right) + 4$$

Step 2: Perform division $\frac{27}{9}$

$$\frac{27}{9} = 3$$

Expression becomes:

$$7 - 2 \times 3 + 4$$

Step 3: Perform multiplication 2×3

$$2 \times 3 = 6$$

Expression becomes:

$$7 - 6 + 4$$

Step 4: Perform subtraction and addition from left to right

$$7 - 6 = 1$$

$$1 + 4 = 5$$

Final value:

$$A = 5$$

8. Explain comparison operators in Python with examples.

Ans: Comparison operators, also known as relational operators, are used to compare two values. They return a Boolean value: True if the condition is met, and False otherwise. These operators are commonly used in decision-making and loops.

1. Equal to (==)

The equality operator checks if two values are exactly the same.

Example:

```
a=5
b=4
c=5
print(a==b)
print(a==c)
```

Output:

```
False
True
```

2. Not Equal to (!=):

The inequality operator checks if two values are not equal.

Example:

```
a=5
b=4
c=5
print(a!=b)
print(a!=c)
```

Output:

```
True
False
```

3. Greater Than Operator (>):

This operator checks if the left operand is larger than the right.

Example:

```
a=5
b=4
print(a>b)
```

Output:

True

4. Less Than Operator (<):

It checks if the left operand is smaller than the right.

Example:

```
a=5  
b=4  
print(a<b)
```

Output:

False

5. Greater Than or Equal To Operator (>=)

It checks if the left operand is greater than or equal to the right.

Example:

```
a=5  
b=4  
print(a>=b)
```

Output: True

6. Less Than or Equal To Operator (<=)

It checks if the left operand is less than or equal to the right.

Example:

```
a=5  
b=4  
print(a<=b)
```

Output: False

9. Write syntax for if-else and Write a Python program to find the largest of two numbers using if-else.

Ans:

The simple **if-else** statement is the most basic form of conditional logic in Python. It allows your program to choose between two outcomes: executing one block of code if a condition is true, or executing a different block of code if the condition is false.

Syntax:

```
if <condition_is_true>:
    # Code Block 1: Executes only if the condition is True.
    statement_1
else:
    # Code Block 2: Executes only if the condition is False.
    statement_2
```

Program to find the largest of two numbers using if-else:

Program:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if (a>b):
    print(a, f"is the largest number than {b}")
else:
    print(b, f"is the largest number than {a}")
```

Output:

```
Enter first number: 12
Enter second number: 13
13 is the largest number than 12
```

```
Enter first number: 4
Enter second number: 3
4 is the largest number than 3
```

10. Explain string counting and searching methods with examples.

Ans:

String counting method:

It is possible to find out how many times a string is repeated in another string. This is done using the count() operation.

for example,

```
my_string = 'Count the number of spaces'
print("my_string.count(' '):", my_string.count(' '))
```

```
my_string.count(' '): 4
```

Searching methods (finding sub strings):

You can find out if one string is a substring of another string using the find() method. This method takes a second string as a parameter and checks to see if that string is in the string receiving the find() method, for example:

string.find(string to find)

The method returns -1 if the string is not present.

```
my_string = 'Count the number of spaces'
print(my_string.find('zero'))
```

```
-1
```

Otherwise, it returns an index indicating the start of the substring.

For example,

```
my_string = 'Count the number of spaces'
print(my_string.find('the'))
```

```
6
```

This prints out the value 6 (the index of the first letter of the substring “the” note strings are indexed from Zero; thus the first letter is at position Zero, the second at position one etc.

11. Write a python program to compare three numbers using elif statement.

Ans:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

# Comparing the numbers
if a > b and a > c:
    print(a, f"is the largest number than {b} and {c}")
elif b > a and b > c:
    print(b, f"is the largest number than {a} and {c}")
elif c > a and c > b:
    print(c, f"is the largest number than {a} and {b}")
else:
    print("Two or more numbers are equal and largest.")
```

Output:

```
Enter first number: 10
Enter second number: 9
Enter third number: 8
10 is the largest number than 9 and 8
```

```
Enter first number: 10
Enter second number: 10
Enter third number: 5
Two or more numbers are equal and largest.
```

12. Write a program to print factorial of a given number using recursive function.

Ans:

Recursion is a process where a function calls itself to solve a problem. A recursive function breaks down a complex problem into smaller, simpler problems. It continues calling itself until it reaches a condition that stops the recursion.

Program:

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial(n - 1)  
  
n = int(input("Enter a number: "))  
if n < 0:  
    print("Factorial is not defined for negative numbers.")  
else:  
    print(f"Factorial of {n} is {factorial(n)}")
```

Output:

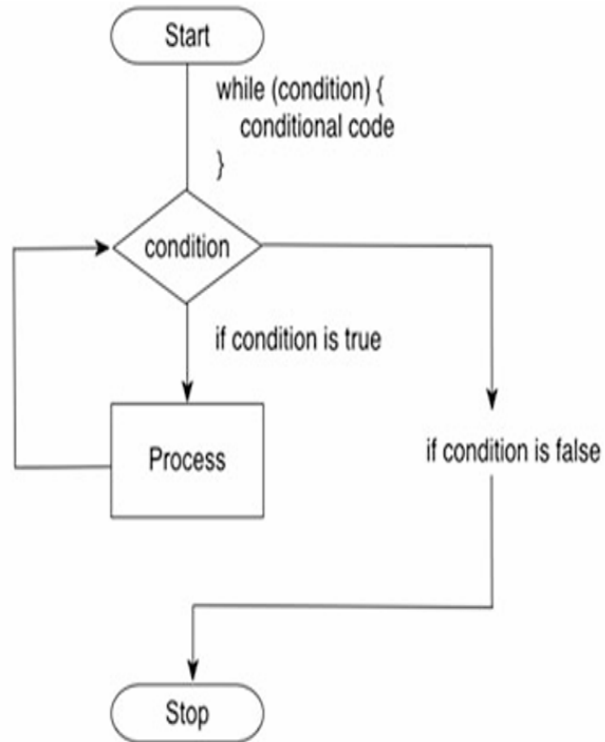
```
Enter a number: 6  
Factorial of 6 is 720
```

```
Enter a number: -3  
Factorial is not defined for negative numbers.
```

13. Explain briefly about while loop and write a program to print the sum of first 10 natural numbers using a while loop.

Ans:

- The **while** loop exists in almost all programming languages and is used to **iterative** (or repeat) one or more code statements as long as the test condition (expression) is **True**.
- This iteration construct is usually used when the number of times we need to repeat the block of code to execute is **not known**.



- The Python while loop has the basic form:

Syntax: **while** <test-condition-is-true>:
statement or statements

Program to print the sum of first 10 natural numbers using a while loop:

Program:

```
n=10
i=1
sum=0
while(i<=n):
    sum=sum+i
    i+=1
print(sum)
```

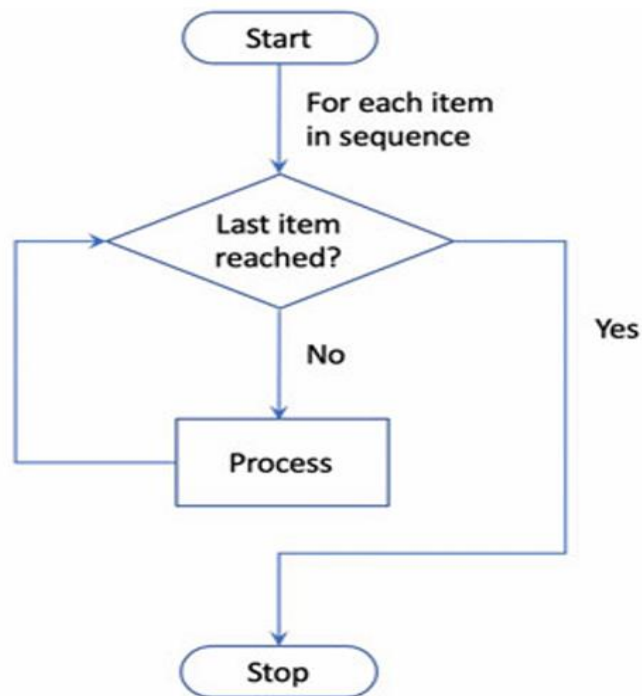
Output:

55

14. Discuss in detail about for loop and Write a program to check whether the given number is prime number or not using for loop.

Ans:

The for loop is used to step a variable through a series of values until a given test is met.



- In Python the approach is slightly different as the values from 0 up to 10 are represented by a **range**.
- The format of the Python for loop when using a range of values is.

Syntax: **for <variable-name> in range(...):**
 statement
 statement

Program to check whether the given number is prime number or not using for loop.

Program:

```
n= int(input("enter a number: "))
for i in range(2,n):
    if (n%i==0):
        print("The given number is not prime")
        break
    else:
        print("The given number is prime")
```

Output:

```
enter a number: 7
The given number is prime

enter a number: 6
The given number is not prime
```

15. Write a Python program to print numbers from 0 to 4 but skip number 3 using loop control statements.

Ans: Program:

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

```
0
1
2
4
```

Explanation:

- The **for** loop iterates over numbers 0 through 4.
- When i equals 3, the continue statement causes the loop to skip printing 3 and move to the next iteration.
- All other numbers (0, 1, 2, 4) are printed.

16. Write a Python program to check whether the given number is Palindrome or not.

Ans:

Program:

```
n=int(input("enter a number"))
i=n
sum=0
while n>0:
    a=n%10
    sum=sum*10+a
    n=n//10
if (sum==i):
    print("The given number is palindrome")
else:
    print("The given number is not a palindrome")
```

Output:

```
enter a number 121
The given number is palindrome
```

```
enter a number 123
The given number is not a palindrome
```

17. Explain briefly about Sorting and Merging lists with examples.

Ans: Sorting and Merging Lists:

SORTING LISTS:

▪ **sort() method:**

The **sort()** method sorts a list in-place in ascending order by default. This method changes the original list.

Example:

```
X = [54, 12, 85, 65, 74, 90]
X.sort()
print("Sorted list:",X)
```

Output:

Sorted list: [12, 54, 65, 74, 85, 90]

▪ reverse() method:

To sort a list in descending order, set the **reverse parameter of sort() to True**.

Example:

```
X = [54, 12, 85, 65, 74, 90]
X.sort(reverse=True)
print("sorted list:",X)
```

Output:

sorted list: [90, 85, 74, 65, 54, 12]

▪ sorted() method:

sorted() returns a new sorted list, leaving the original list unchanged. It is ideal when you need a temporary sort.

Example:

```
X = [54, 12, 85, 65, 74, 90]
Y=sorted(X)
print("original list",X)
print("sorted list:",Y)
```

Output:

original list [54, 12, 85, 65, 74, 90]
sorted list: [12, 54, 65, 74, 85, 90]

▪ Custom Sorting with key Parameter

You can define custom sort criteria using the key parameter, which accepts a function to customize the sort order.

Example:

```
fruit = ["banana", "apple", "kiwi"]
fruit.sort(key=len)
print(fruit)
```

Output:

['kiwi', 'apple', 'banana']

MERGING LISTS:

Using + operator:

It is possible to concatenate two lists together using the concatenation operator '+'.

Example:

```
data1=['dog', 'cat', 'fish']  
data2=[1,2,3,4]  
data3=data1+data2  
print(data3)
```

Output:

```
['dog', 'cat', 'fish', 1, 2, 3, 4]
```

Using extend():

Another common method is to use the **extend()** function, which modifies the original list by adding elements from another list.

Example:

```
data1=['dog', 'cat', 'fish']  
data2=[1,2,3,4]  
data1.extend(data2)  
print(data1)
```

Output:

```
['dog', 'cat', 'fish', 1, 2, 3, 4]
```

18. Explain briefly about any 5 tuple operations with an example.

Ans:

OPERATIONS ON TUPLES:

▪ Accessing elements of a tuple:

The elements of a Tuple can be accessed using an index in square brackets. The index returns the object at that position.

Example:

```
Tup1 = (1, 2, 3)  
print('Tup1[0]:', Tup1[0])  
print('Tup1[1]:\t', Tup1[1])
```

Output:

```
Tup1[0]: 1
```

```
Tup1[1]:    2
```

- **Negative Indexing:**

You can also use negative indices to access elements from the end of the tuple.

Example:

```
Tup1 = (1, 'abc',2.45,5)
print(Tup1[-2])
```

Output:

```
2.45
```

- **Slicing:**

Slicing is done by tuple(iterable) providing the start and end indexes for the slice, separated by a colon, within the index square brackets.

➤ **For example:**

```
Tup1 = (1, 'abc',2.45,5)
print(Tup1[0:3])
print(Tup1[1:])
print(Tup1[:2])
```

Output:

```
(1, 'abc', 2.45)
('abc', 2.45, 5)
(1, 'abc')
```

- **Reversing a Tuple:**

You can reverse a Tuple using the `:: - 1` notation (again this returns a new Tuple and has no effect on the original Tuple):

Example:

```
Tup1 = (1, 'abc',2.45,5)
print(Tup1[::-1])
```

Output:

```
(5, 2.45, 'abc', 1)
```

- **Concatenation:**

You can combine two or more tuples using the `+` operator. This operation creates a new tuple.

Example:

```
Tup1=(3,6,8,2)
Tup2=('xyz',3.2,1)
Tup3=Tup1+Tup2
print(Tup3)
```

Output:

```
(3, 6, 8, 2, 'xyz', 3.2, 1)
```

▪ Repetition:

The * operator can be used to repeat a tuple a specified number of times, also creating a new tuple.

Example:

```
Tup=(3,6,8,2)
print(Tup*2)
```

Output:

```
(3, 6, 8, 2, 3, 6, 8, 2)
```

▪ Comparision:

If all the elements in both the tuples are same then it returns True, otherwise it returns False.

Example:

```
Tup1=(3,6,8,2)
Tup2=('xyz',3,1)
print(Tup1==Tup2)
print(Tup1!=Tup2)
```

Output:

```
False
True
```

19. Discuss about any five tuple built- in functions in python with example.**Ans:**

- **count():** Returns the number of times a specified value appears in the tuple.

Example:

```
num = (10, 20, 5, 30, 15, 5)
print("Count of 5 is ",num.count(5))
```

Output:

Count of 5 is 2

- **index():** Searches the tuple for a specified value and returns the index of its first occurrence. If the value is not found, it raises a ValueError.

Example:

```
num = (10, 20, 5, 30, 15, 5)
print("Index of 5 is ", num.index(5))
```

Output:

Index of 5 is 2

- **len():** Returns the number of elements in the tuple.

Example:

```
numbers = (10, 20, 5, 30, 15)
print("Length:", len(numbers))
```

Output:

Length: 5

- **max():** Returns the largest item in the tuple (if all items are of the same comparable type).

Example:

```
numbers = (10, 20, 5, 30, 15)
print("Maximum:", max(numbers))
```

Output:

Maximum: 30

- **min():** Returns the smallest item in the tuple (if all items are of the same comparable type).

Example:

```
numbers = (10, 20, 5, 30, 15)
print("Minimum:", min(numbers))
```

Output:

Minimum: 5

- **sum():** Returns the sum of all items in the tuple (if they are all numbers).

Example:

```
numbers = (10, 20, 5, 30, 15)
```

```
print("Sum:",sum(numbers))
```

Output:

Sum: 80

20.Explain how to create a list and a list of lists in Python with examples.

Ans: A **list** in Python is an ordered, mutable collection of items enclosed in square brackets [], with elements separated by commas. Lists can hold items of any type, including numbers, strings, or even mixed types.

```
# List of integers
numbers = [1, 2, 3, 4, 5]

# List of strings
fruits = ['apple', 'banana', 'cherry']

# List with mixed types
mixed = [1, 'hello', 3.14, True]

print(numbers)
print(fruits)
print(mixed)

[1, 2, 3, 4, 5]
['apple', 'banana', 'cherry']
[1, 'hello', 3.14, True]
```

Creating a List of Lists in Python

A *list of lists* is simply a list where each element is itself a list (i.e., nested lists).

```
# List of lists (matrix)
```

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

```
print(matrix)
```

```
# Access an element, e.g., row 1, column 2
```

```
print(matrix[1][2]) # Output: 6
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
6
```